



TP D'INTRODUCTION À LA PROGRAMMATION ASSEMBLEUR 68000 AVEC EASy68K

1. RAPPELS SUR LE MICROPROCESSEUR 68000

1.1 Introduction

Le Motorola 68000 est un microprocesseur 16/32 bits introduit en 1979. Il possède un bus de données de 16 bits et un bus d'adresses de 24 bits, permettant d'adresser jusqu'à 16 Mo de mémoire[2]. Bien que son bus externe soit de 16 bits, les registres internes sont de 32 bits, ce qui explique sa classification comme processeur 16/32 bits.

1.2 Caractéristiques du 68000

Le 68000 possède 18 registres :

- 1 registre Compteur Ordinal sur 24 bits : PC (Program Counter)
- 1 registre d'état sur 16 bits : SR (Status Register)
- 8 registres de données sur 32 bits : D0 – D7
- 7 registres d'adresses sur 32 bits : A0 – A6
- 1 registre pointeur de pile sur 32 bits : A7 (Stack Pointer)

Registres de données (D0-D7) : Ces registres sont l'équivalent des accumulateurs, c'est-à-dire ce sont des registres sur lesquels on peut effectuer des opérations arithmétiques. Celles-ci ont de plus la particularité d'affecter le registre d'état (SR).

Registres d'adresses (A0-A6) : Ces registres sont utilisés principalement pour stocker des adresses mémoire et peuvent servir de pointeurs de base ou de pointeurs de pile.

Registre A7 (Stack Pointer) : Ce registre sert de pointeur de pile et joue un rôle crucial dans la gestion des sous-programmes et des interruptions.

1.3 Modes de fonctionnement

Le 68000 possède deux modes de fonctionnement : le mode utilisateur et le mode superviseur[4].

Mode superviseur : C'est le mode dans lequel se trouve le microprocesseur lorsqu'il exécute un programme d'exception suite à une interruption générée de manière interne (erreur d'adresse, erreur de bus, etc.) : deux événements que l'on regroupe généralement sous le vocable d'EXCEPTION. Dans ce mode, le microprocesseur dispose de nouvelles ressources : il travaille avec une nouvelle pile, la pile super-utilisateur, ainsi que d'un nouveau registre d'état. De plus, il peut exécuter dans ce mode un certain nombre d'instructions spéciales.

Mode utilisateur : C'est le mode dans lequel se trouve le microprocesseur lorsqu'il exécute un programme "normal". Il utilise la pile et le registre d'état dit "utilisateur". Dans ce mode, les instructions privilégiées ne sont pas exécutables.

1.4 Format des données

Si le microprocesseur peut travailler sur des données de 8, 16 ou 32 bits à condition de préciser le format des instructions (format .B .W .L), par contre une adresse est toujours exprimée sur 32 bits. Par défaut, le format est de 16 bits.

Suffixe	Taille	Description
.B	8 bits	Byte (octet)
.W	16 bits	Word (mot) - format par défaut
.L	32 bits	Long word (mot long)

Table 1: Formats de données du 68000

Le bus de données comprend 16 bits et peut donc transférer un mot de 16 bits en une seule fois (le 68000 est un microprocesseur 16 bits). Pour rester compatible avec la nombreuse famille des composants 8 bits, il se décompose en deux parties :

- La partie MSB (Most Significant Byte) pour les 8 bits de poids fort (bits 15-8)
- La partie LSB (Least Significant Byte) pour les 8 bits de poids faible (bits 7-0)

Organisation mémoire : Le 68000 utilise l'ordre des octets big-endian, ce qui signifie que l'octet de poids fort (MSB) est stocké à l'adresse la plus basse. Par exemple, pour un mot de 16 bits à l'adresse \$1000, l'octet de poids fort sera à l'adresse \$1000 et l'octet de poids faible à l'adresse \$1001.

1.5 Registre d'état (SR)

Le registre d'état (Status Register) de 16 bits est divisé en deux parties :

- **Octet système** (bits 15-8) : contient les informations liées au système (masque d'interruption, mode superviseur/utilisateur, bit de trace)
- **Octet utilisateur** (bits 7-0) : contient les indicateurs de condition (CCR - Condition Code Register)

Indicateurs de condition principaux :

Bit	Nom	Description
X	Extend	Extension pour opérations multi-précision
N	Negative	Résultat négatif (bit de signe)
Z	Zero	Résultat nul
V	Overflow	Débordement arithmétique
C	Carry	Retenue ou emprunt

2. PRISE EN MAIN D'EASy68K

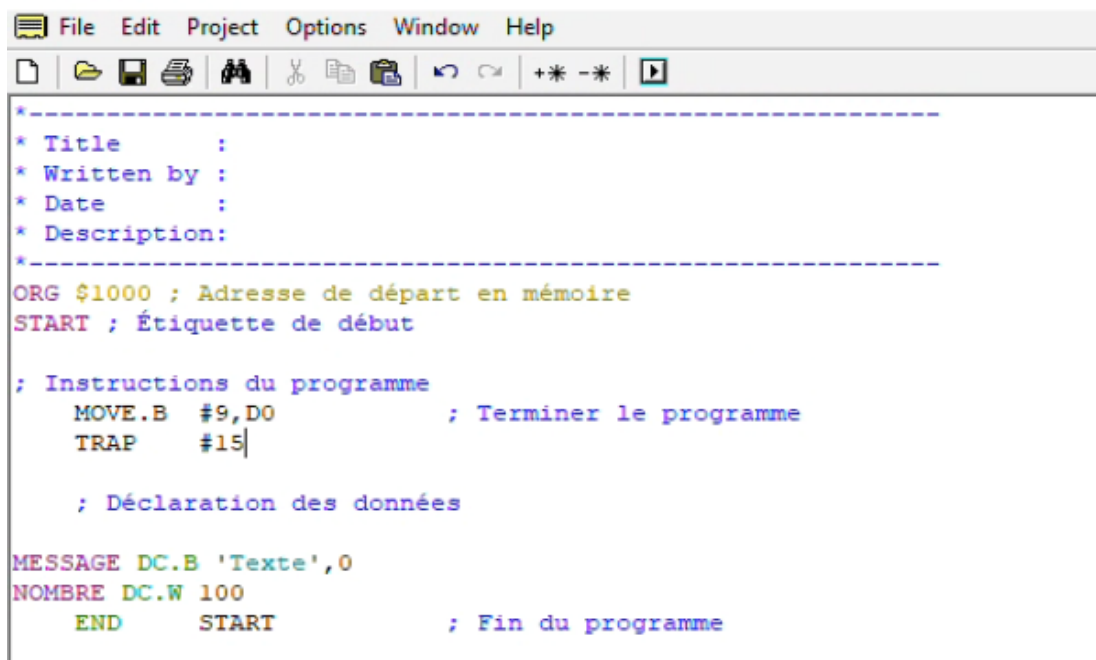
2.1 Interface du simulateur

EASy68K est un environnement de développement intégré pour la programmation assembleur du 68000. Il comprend :

- Un éditeur de texte avec coloration syntaxique
- Un assembleur (convertit le code source en code machine)
- Un simulateur (exécute le code machine)
- Un débogueur (permet l'exécution pas-à-pas)

2.2 Structure d'un programme assembleur

Tout programme assembleur 68000 respecte la structure suivante :

The screenshot shows the EASy68K IDE interface. The menu bar includes File, Edit, Project, Options, Window, and Help. The toolbar contains icons for file operations (new, open, save, print), editing (undo, redo, cut, copy, paste), and execution (run). The main text area displays an assembly program with the following structure:

```
*-----*
* Title      :
* Written by :
* Date       :
* Description:
*-----*
ORG $1000 ; Adresse de départ en mémoire
START ; Étiquette de début

; Instructions du programme
    MOVE.B #9,D0          ; Terminer le programme
    TRAP    #15|

    ; Déclaration des données

MESSAGE DC.B 'Texte',0
NOMBRE DC.W 100
    END      START          ; Fin du programme
```

- **Éléments clés :**
- **Commentaires :** Lignes commençant par * ou ;
- **ORG :** Définit l'adresse de départ du programme en mémoire (généralement \$1000)
- **Étiquettes :** Noms symboliques pour les adresses (ex: START, MESSAGE)
- **Instructions :** Opérations à exécuter (MOVE, ADD, TRAP, etc.)
- **Directives :** DC.B, DC.W, DC.L pour déclarer des données
- **END :** Marque la fin du programme avec l'adresse de départ

2.3 TRAP #15 - Entrées/Sorties

TRAP #15 est l'instruction système utilisée pour effectuer des entrées/sorties dans le simulateur EASy68K. Le numéro de tâche est placé dans le registre Do.

Tâche	Description	Paramètres
0	Afficher chaîne (longueur) + CR,LF	A1=adresse, D1.W=longueur
1	Afficher chaîne (longueur) sans CR,LF	A1=adresse, D1.W=longueur
3	Afficher nombre signé en décimal	D1.L=nombre
4	Lire un nombre au clavier	→ D1.L
13	Afficher chaîne terminée par o + CR,LF	A1=adresse
14	Afficher chaîne terminée par o sans CR,LF	A1=adresse
9	Terminer le programme	-

3. EXERCICES PRATIQUES

Exercice 3.1 : Premier programme "Hello World"

Objectif : Afficher un message à l'écran et se familiariser avec l'environnement.

ORG \$1000 ; Origine du programme

START LEA MESSAGE,A1 ; A1 = adresse du message

MOVE.B #14,Do ; Tâche 14 : afficher chaîne terminée par o

TRAP #15 ; Appel système d'E/S

MOVE.B #9,D0 ; Tâche 9 : terminer le programme
TRAP #15

MESSAGE DC.B 'Bonjour! Bienvenue en assembleur 68000',o

END START

Travail à réaliser :

1. Tapez ce programme dans EASy68K
2. Assemblez-le avec F9 ou le bouton "Assemble"
3. Exécutez-le avec F10 ou le bouton "Execute"
4. Observez le résultat dans la fenêtre de sortie

Questions :

1. Que fait l'instruction LEA MESSAGE, A1 ?
2. Pourquoi utilise-t-on la tâche 14 plutôt que la tâche 0 ?

3. Que signifie le 0 à la fin de la chaîne MESSAGE ?
4. Modifiez le message et testez à nouveau le programme

Exercice 3.2 : Manipulation des registres

Objectif : Comprendre les registres de données Do-D7 et l'exécution pas-à-pas.

*-----

- Programme : Manipulation de registres
- Description : Copie de valeurs entre registres

*-----

```
ORG $1000

START MOVE.L #100,Do ; Do = 100 (décimal)
MOVE.L #$FF,D1 ; D1 = 255 (hexadécimal)
MOVE.L Do,D2 ; D2 = copie de Do
```

```
ADD.L    D1,D2          ; D2 = D2 + D1

MOVE.L   D2,D3          ; D3 = résultat

MOVE.B   #9,D0          ; Terminer
TRAP     #15

END      START
```

Travail à réaliser :

1. Exécutez le programme en mode pas-à-pas (touche F8)
2. Observez la fenêtre des registres après chaque instruction
3. Notez les valeurs de Do, D1, D2, D3 à chaque étape
4. Complétez le tableau ci-dessous

Instruction	Do	D1	D2	D3
Après MOVE.L #100,Do				
Après MOVE.L #\$FF,D1				
Après MOVE.L Do,D2				
Après ADD.L D1,D2				
Après MOVE.L D2,D3				

Questions :

1. Quelle est la valeur finale de D3 en décimal ?

2. Quelle est la différence entre MOVE.L, MOVE.W et MOVE.B ?
3. Testez avec d'autres valeurs et vérifiez vos résultats

Exercice 3.3 : Affichage de nombres

Objectif : Utiliser TRAP #15 tâche 3 pour afficher un nombre calculé.

*-----

- Programme : Affichage de nombre
- Description : Calcule et affiche une somme

*-----

ORG \$1000

START MOVE.L #25,D1 ; Premier nombre

ADD.L #17,D1 ; D1 = 25 + 17 = 42

```
MOVE.B #3,D0          ; Tâche 3 : afficher D1 en décimal
TRAP   #15

MOVE.B #9,D0          ; Terminer
TRAP   #15

END     START
```

Travail à réaliser :

1. Exécutez le programme et vérifiez le résultat (42)
2. Modifiez les valeurs 25 et 17 pour tester d'autres additions
3. Remplacez ADD.L par SUB.L (soustraction) et testez
4. Testez avec MULU.W (multiplication non signée sur 16 bits)

Question : Quelle est la différence entre MULU (multiplication non signée) et MULS (multiplication signée) ?

Exercice 3.4 : Lecture au clavier

Objectif : Interagir avec l'utilisateur en lisant des données au clavier.

*-----

- Programme : Lecture de nombre
- Description : Lit un nombre et l'affiche

*-----

ORG \$1000

```
START LEA PROMPT,A1 ; Message de demande
MOVE.B #14,D0 ; Afficher le prompt
TRAP #15
```

```
MOVE.B #4,D0 ; Tâche 4 : lire un nombre dans D1
TRAP #15 ; Le nombre lu est dans D1.L

LEA RESULT,A1 ; Message résultat
MOVE.B #14,D0
TRAP #15

MOVE.B #3,D0 ; Afficher le nombre lu
TRAP #15

MOVE.B #9,D0 ; Terminer
TRAP #15
```

```
PROMPT DC.B 'Entrez un nombre : ',0
RESULT DC.B 'Vous avez saisi : ',0
```

```
END START
```

Travail à réaliser :

1. Exécutez et testez avec différents nombres positifs
2. Testez avec un nombre négatif
3. Testez avec 0
4. Modifiez les messages PROMPT et RESULT

Exercice 3.5 : Calculatrice simple (Addition)

Objectif : Combiner lecture, calcul et affichage.

*_____

- Programme : Calculatrice - Addition
- Description : Lit deux nombres et affiche leur somme

```
*_____
ORG $1000
```

```
START LEA MSG1,A1 ; Premier message
MOVE.B #14,D0
TRAP #15
```

```
MOVE.B #4,D0 ; Lire premier nombre
TRAP #15
MOVE.L D1,D2 ; Sauvegarder dans D2

LEA MSG2,A1 ; Deuxième message
```



```

MOVE.B  #14,D0
TRAP    #15

MOVE.B  #4,D0      ; Lire deuxième nombre
TRAP    #15        ; Nombre dans D1

ADD.L   D2,D1      ; D1 = D1 + D2 (somme)

LEA     MSG3,A1    ; Message résultat
MOVE.B  #14,D0
TRAP    #15

MOVE.B  #3,D0      ; Afficher la somme
TRAP    #15

MOVE.B  #9,D0      ; Terminer
TRAP    #15

```

```

MSG1 DC.B 'Premier nombre : ',0
MSG2 DC.B 'Deuxieme nombre : ',0
MSG3 DC.B 'Somme = ',0

```

```

END      START

```

Travail à réaliser :

1. Testez le programme avec : 10 + 20
2. Testez avec des nombres négatifs : -5 + 8
3. Créez une version pour la soustraction en remplaçant ADD.L par SUB.L
4. Créez une version pour la multiplication avec MULS.W

Exercice 3.6 : Utilisation de la tâche 0 et 1

Objectif : Comprendre la différence entre l'affichage avec et sans retour à la ligne.

Programme avec tâche 0 (avec CR, LF) :

*_-----

- Programme : Affichage avec tâche 0
- Description : Affiche une chaîne avec retour à la ligne
*_-----
ORG \$1000

```

START LEA MSG,A1 ; A1 = adresse de la chaîne
MOVE.W #13,D1 ; Longueur = 13 caractères
MOVE.B #0,D0 ; Tâche 0 : afficher avec CR, LF
TRAP #15

```

```

MOVE.B #9,D0          ; Terminer
TRAP    #15

```

MSG DC.B 'Bonjour monde'

```

END      START

```

Programme avec tâche 1 (sans CR, LF) :

*-----

- Programme : Affichage avec tâche 1
- Description : Affiche deux chaînes sur la même ligne

*-----

ORG \$1000

```

START LEA MSG1,A1 ; Première chaîne
MOVE.W #8,D1 ; Longueur = 8
MOVE.B #1,D0 ; Tâche 1 : afficher sans CR, LF
TRAP #15

```

```

LEA    MSG2,A1          ; Deuxième chaîne
MOVE.W #6,D1            ; Longueur = 6
MOVE.B #1,D0            ; Tâche 1 : afficher sans CR, LF
TRAP    #15

MOVE.B #9,D0          ; Terminer
TRAP    #15

```

MSG1 DC.B 'Bonjour '

MSG2 DC.B 'monde!'

```

END      START

```

Travail à réaliser :

1. Exécutez les deux programmes et observez la différence
2. Notez que la tâche 0 et 1 nécessitent de spécifier la longueur dans D1.W
3. Modifiez le deuxième programme pour afficher trois mots sur la même ligne

4. INSTRUCTIONS DE BRANCHEMENT ET BOUCLES

4.1 Instructions de comparaison et branchement

Le 68000 utilise l'instruction CMP pour comparer deux valeurs, puis des instructions de branchement conditionnel pour modifier le flux d'exécution[11].

Instruction	Condition	Description
BEQ label	Z=1	Brancher si égal (Equal)
BNE label	Z=0	Brancher si différent (Not Equal)
BGT label	Z=0 ET N=V	Brancher si plus grand (Greater Than, signé)
BLT label	N≠V	Brancher si plus petit (Less Than, signé)
BGE label	N=V	Brancher si plus grand ou égal (Greater or Equal)
BLE label	Z=1 OU N≠V	Brancher si plus petit ou égal (Less or Equal)
BRA label	-	Brancher toujours (inconditionnelle)

4.2 Exercice : Comparaison de deux nombres

Objectif : Déterminer quel nombre est le plus grand.

*-----

- Programme : Comparaison
- Description : Compare deux nombres et affiche le résultat

*-----

ORG \$1000

START MOVE.L #15,Do ; Premier nombre

MOVE.L #23,D1 ; Deuxième nombre

```

CMP.L    D1,D0          ; Comparer D0 avec D1 (D0 - D1)
BGT      D0_PLUS_GRAND  ; Si D0 > D1
BLT      D1_PLUS_GRAND  ; Si D0 < D1

; Cas où D0 = D1
LEA      MSG_EGAL,A1
MOVE.B   #14,D0
TRAP     #15
BRA      FIN

```

Do_PLUS_GRAND

LEA MSG_Do,A1

MOVE.B #14,Do

TRAP #15

BRA FIN

D1_PLUS_GRAND

LEA MSG_D1,A1

MOVE.B #14,Do

TRAP #15

```
FIN MOVE.B #9,D0
TRAP #15
```

```
MSG_Do DC.B 'Le premier nombre est plus grand',0
MSG_D1 DC.B 'Le deuxieme nombre est plus grand',0
MSG_EGAL DC.B 'Les deux nombres sont egaux',0
```

```
END      START
```

Travail à réaliser :

1. Testez avec D0=15 et D1=23 (résultat attendu : D1 plus grand)
2. Testez avec D0=30 et D1=20 (résultat attendu : D0 plus grand)
3. Testez avec D0=10 et D1=10 (résultat attendu : égaux)
4. Modifiez pour lire les deux nombres au clavier

4.3 Exercice : Boucle simple

Objectif : Créer une boucle qui compte de 1 à 10.

*-----

- Programme : Boucle de comptage
- Description : Affiche les nombres de 1 à 10

*-----

```
ORG $1000
```

```
START MOVE.L #1,D2 ; Compteur = 1
```

```
BOUCLE MOVE.L D2,D1 ; D1 = valeur à afficher
```

```
MOVE.B #3,D0 ; Tâche 3 : afficher nombre
```

```
TRAP #15
```

```
LEA     ESPACE,A1      ; Afficher un espace
MOVE.B  #14,D0
TRAP    #15

ADD.L   #1,D2           ; Compteur++
CMP.L   #11,D2          ; Comparer avec 11
BNE     BOUCLE          ; Si != 11, continuer

MOVE.B  #9,D0           ; Terminer
TRAP    #15
```

```
ESPACE DC.B ' ',0
```

```
END      START
```

Travail à réaliser :

1. Exécutez le programme et vérifiez qu'il affiche : 1 2 3 4 5 6 7 8 9 10
2. Modifiez pour afficher de 0 à 20
3. Modifiez pour afficher de 10 à 1 (compte à rebours)

5. MANIPULATION DE TABLEAUX

5.1 Adressage indirect avec registres d'adresse

Les registres d'adresse A0-A6 sont utilisés comme pointeurs pour accéder à la mémoire. Le mode d'adressage indirect permet d'accéder aux données pointées par un registre d'adresse.

Notation	Description	Exemple
(An)	Adresse contenue dans An	MOVE.W (A0),D1
(An)+	Post-incrément (An puis An=An+taille)	MOVE.W (A0)+,D1
-(An)	Pré-décrément (An=An-taille puis An)	MOVE.W -(A0),D1
d(An)	An + déplacement d	MOVE.W 4(A0),D1

5.2 Exercice : Somme d'un tableau

Objectif : Calculer la somme des éléments d'un tableau.

*-----

- Programme : Somme d'un tableau
- Description : Calcule la somme des éléments d'un tableau

*-----

```
ORG $1000

START LEA TABLEAU,A0 ; A0 pointe vers le tableau
MOVE.W #5,D2 ; Compteur (5 éléments)
CLR.L D1 ; D1 = 0 (somme)
```

```
BOUCLE ADD.W (A0)+,D1 ; D1 = D1 + *A0, puis A0++
SUB.W #1,D2 ; D2 = D2 - 1
BNE BOUCLE ; Si D2 != 0, continuer
```

```
LEA    MSG,A1          ; Message
MOVE.B #14,D0
TRAP   #15

MOVE.B #3,D0           ; Afficher la somme
TRAP   #15

MOVE.B #9,D0           ; Terminer
```

TRAP #15

TABLEAU DC.W 10,20,30,40,50 ; Tableau de 5 mots (words)
MSG DC.B 'Somme du tableau = ',o

END START

Travail à réaliser :

1. Exécutez le programme (résultat attendu : 150)
2. Modifiez les valeurs du tableau et vérifiez le résultat
3. Ajoutez un 6ème élément au tableau et modifiez le compteur
4. Expliquez le fonctionnement de l'instruction `ADD.W (A0)+,D1`

Questions :

1. Que fait l'instruction `(A0)+` ?
2. Pourquoi utilise-t-on `ADD.W` plutôt que `ADD.L` ?
3. Quelle est la valeur de `A0` à la fin de la boucle ?

5.3 Exercice : Trouver le maximum

Objectif : Parcourir un tableau et trouver le plus grand élément.

*-----

- Programme : Maximum d'un tableau
- Description : Trouve la plus grande valeur

*-----

ORG \$1000

START LEA TABLEAU,A0 ; A0 pointe vers le tableau
MOVE.W (A0)+,D1 ; D1 = premier élément (max actuel)
MOVE.W #4,D2 ; 4 éléments restants

BOUCLE MOVE.W (A0)+,D3 ; D3 = élément suivant
CMP.W D1,D3 ; Comparer D3 avec D1 (max)
BLE SUIVANT ; Si D3 <= D1, passer au suivant
MOVE.W D3,D1 ; Sinon, D1 = D3 (nouveau max)

SUIVANT SUB.W #1,D2 ; D2--
BNE BOUCLE ; Continuer si D2 != 0

LEA MSG,A1 ; Message résultat
MOVE.B #14,D0
TRAP #15

MOVE.B #3,D0 ; Afficher le maximum
TRAP #15

```
MOVE.B #9,D0 ; Terminer
TRAP #15
```

TABLEAU DC.W 15,42,8,99,23 ; Tableau de 5 mots
MSG DC.B 'Maximum = ',0

```
END START
```

Travail à réaliser :

1. Exécutez le programme (résultat attendu : 99)
2. Modifiez les valeurs du tableau pour tester
3. Créez une version qui trouve le minimum (BLE devient BGE)
4. Créez une version qui compte les nombres pairs dans le tableau

6. CALCULATRICE COMPLÈTE

Objectif : Créer une calculatrice avec menu et plusieurs opérations.

Cahier des charges :

1. Afficher un menu avec 4 choix : Addition, Soustraction, Multiplication, Division
2. Lire le choix de l'utilisateur
3. Lire deux nombres
4. Effectuer l'opération choisie
5. Afficher le résultat
6. Gérer les choix invalides

Code de départ :

*-----

- Programme : Calculatrice complète
- Description : Addition, soustraction, multiplication, division

*-----

ORG \$1000

START LEA MENU,A1 ; Afficher le menu

MOVE.B #14,D0

TRAP #15

```
MOVE.B #4,D0 ; Lire le choix
TRAP #15
MOVE.L D1,D3 ; Sauvegarder le choix dans D3

LEA MSG1,A1 ; Demander premier nombre
MOVE.B #14,D0
```

```

TRAP    #15

MOVE.B  #4,D0
TRAP    #15
MOVE.L  D1,D4          ; Sauvegarder dans D4

LEA     MSG2,A1        ; Demander deuxième nombre
MOVE.B  #14,D0
TRAP    #15

MOVE.B  #4,D0
TRAP    #15
MOVE.L  D1,D5          ; Sauvegarder dans D5

; Tester le choix
CMP.L   #1,D3          ; Choix = 1 (addition) ?
BEQ     ADDITION

CMP.L   #2,D3          ; Choix = 2 (soustraction) ?
BEQ     SOUSTRACTION

CMP.L   #3,D3          ; Choix = 3 (multiplication) ?
BEQ     MULTIPLICATION

CMP.L   #4,D3          ; Choix = 4 (division) ?
BEQ     DIVISION

LEA     MSG_ERREUR,A1   ; Choix invalide
MOVE.B  #14,D0
TRAP    #15
BRA     FIN

```

ADDITION

```

MOVE.L D4,D1 ; D1 = premier nombre
ADD.L D5,D1 ; D1 = D1 + D5
BRA AFFICHER

```

SOUSTRACTION

```

MOVE.L D4,D1 ; D1 = premier nombre
SUB.L D5,D1 ; D1 = D1 - D5
BRA AFFICHER

```

MULTIPLICATION

```

MOVE.L D4,D1 ; D1 = premier nombre
MULS.W D5,D1 ; D1 = D1 * D5 (signé, 16 bits)
BRA AFFICHER

```

DIVISION

```

MOVE.L D4,D1 ; D1 = dividende
DIVS.W D5,D1 ; D1 = D1 / D5 (quotient dans 16 bits bas)
BRA AFFICHER

```



```

AFFICHER
LEA RESULTAT,A1 ; Message résultat
MOVE.B #14,Do
TRAP #15

```

```

MOVE.B #3,D0 ; Afficher le résultat
TRAP #15

```

```

FIN MOVE.B #9,Do ; Terminer
TRAP #15

```

```

; Données

```

```

MENU DC.B '=== CALCULATRICE ===',13,10
DC.B '1. Addition',13,10
DC.B '2. Soustraction',13,10
DC.B '3. Multiplication',13,10
DC.B '4. Division',13,10
DC.B 'Votre choix : ',0

```

```

MSG1 DC.B 'Premier nombre : ',0
MSG2 DC.B 'Deuxieme nombre : ',0
RESULTAT DC.B 'Resultat = ',0
MSG_ERREUR DC.B 'Choix invalide!',0

```

```

END      START

```

Travail à réaliser :

1. Testez chaque opération (1 à 4)
2. Testez avec un choix invalide (ex: 5)
3. Améliorations possibles :
 - Vérifier la division par zéro avant DIVS
 - Afficher le reste de la division (mot haut de D1 après DIVS)

ANNEXE A : AIDE-MÉMOIRE

A.1 Instructions de base

Instruction	Description	Exemple
MOVE src,dest	Copier src vers dest	MOVE.L Do,D1
LEA src,An	Charger adresse effective	LEA MSG,Ao
ADD src,dest	dest = dest + src	ADD.L D1,D2
SUB src,dest	dest = dest - src	SUB.L #1,D3

MULS src,dest	Multiplication signée (16 bits)	MULS.W D1,D2
MULU src,dest	Multiplication non signée (16 bits)	MULU.W D1,D2
DIVS src,dest	Division signée (16 bits)	DIVS.W D1,D2
DIVU src,dest	Division non signée (16 bits)	DIVU.W D1,D2
CLR dest	Mettre à zéro	CLR.L Do
CMP src,dest	Comparer (dest - src)	CMP.L Do,D1
EXT dest	Étendre le signe	EXT.L Do

A.2 Instructions de branchement

Instruction	Condition	Description
BEQ label	Z=1	Brancher si égal
BNE label	Z=0	Brancher si différent
BGT label	Z=0 ET N=V	Brancher si plus grand (signé)
BLT label	N≠V	Brancher si plus petit (signé)
BGE label	N=V	Brancher si >= (signé)
BLE label	Z=1 OU N≠V	Brancher si <= (signé)
BHI label	C=0 ET Z=0	Brancher si plus grand (non signé)
BLO label	C=1	Brancher si plus petit (non signé)
BRA label	-	Brancher toujours

A.3 TRAP #15 - Tâches complètes

Do	Description	Paramètres
0	Afficher chaîne (longueur) + CR,LF	A1=adresse, D1.W=longueur
1	Afficher chaîne (longueur) sans CR,LF	A1=adresse, D1.W=longueur
2	Lire chaîne (max 80 car.)	A1=buffer, → D1.W=longueur
3	Afficher nombre signé décimal	D1.L=nombre
4	Lire nombre	→ D1.L
5	Lire caractère	→ D1.B
6	Afficher caractère	D1.B=caractère
13	Afficher chaîne terminée par 0 + CR,LF	A1=adresse
14	Afficher chaîne terminée par 0 sans CR,LF	A1=adresse

15	Afficher nombre dans base N	D1.L=nombre, D2.B=base (2-36)
9	Terminer le programme	-

A.4 Directives d'assemblage

Directive	Description	Exemple
ORG	Définir l'adresse d'origine	ORG \$1000
DC.B	Déclarer octet(s)	DC.B 'Hello',o
DC.W	Déclarer mot(s) de 16 bits	DC.W 100,200,300
DC.L	Déclarer mot(s) long de 32 bits	DC.L \$12345678
DS.B	Réserver octet(s)	DS.B 50
DS.W	Réserver mot(s) de 16 bits	DS.W 10
DS.L	Réserver mot(s) long de 32 bits	DS.L 5
EQU	Définir constante	TAILLE EQU 100
END	Marquer la fin	END START

A.5 Codes ASCII utiles

Code décimal	Code hexa	Description
0	\$00	NULL (fin de chaîne)
10	\$0A	LF (Line Feed - saut de ligne)
13	\$0D	CR (Carriage Return - retour chariot)
32	\$20	Espace
48-57	\$30-\$39	Chiffres '0' à '9'
65-90	\$41-\$5A	Lettres 'A' à 'Z'
97-122	\$61-\$7A	Lettres 'a' à 'z'